



PORTFÓLIO + TRILHA DE APRENDIZADO

AWS Data Engineering

Guia Prático de Estudo

Uma jornada em oito sessões para navegar o código, compreender as decisões e apresentar o projeto com evidências.

2.976

linhas na AWS

0

chaves duplicadas

96,6%

menos scan por partition

Pipeline: Open-Meteo -> S3 RAW -> Glue/PySpark -> Parquet CURATED -> Catalog -> Athena

github.com/stanleykaiyzser/aws-data-engineering-learning-lab



Roteiro da jornada

Apostila de acompanhamento. O repositório continua sendo a fonte executável; aqui você encontra a ordem e os checkpoints.

- | | |
|----|---|
| 00 | Como usar, mapa e preparação segura |
| 01 | Arquitetura e narrativa ponta a ponta |
| 02 | Ingestão, RAW, contrato e idempotência |
| 03 | PySpark, qualidade e CURATED |
| 04 | Spark: lazy, partitions, shuffle, broadcast, skew e small files |
| 05 | Parquet, catálogo, particionamento e Athena |
| 06 | Modelagem dimensional e escolha de serviços |
| 07 | Terraform, orquestração, CI e FinOps |
| 08 | Síntese, DataOps e entrevista |
| + | Retenção, navegação rápida e critério de conclusão |

Princípio: não avance por páginas lidas. Avance quando conseguir prever, observar e explicar.

Este é o ponto de entrada para estudar o laboratório. A proposta não é ler todos os arquivos em sequência, mas seguir o dado, executar partes pequenas, observar evidências e explicar as decisões com suas próprias palavras.

Tempo sugerido: **8 sessões de 60 a 90 minutos**. Se tiver menos tempo, faça primeiro as sessões 1, 2, 3, 4 e 8.

Como usar este guia

Em cada sessão, siga sempre o mesmo ciclo:

1. **Prever:** antes de abrir o código, diga o que espera que aconteça.
2. **Inspecionar:** leia somente os arquivos indicados.
3. **Executar ou observar:** rode localmente quando for barato; use as evidências quando a execução AWS teria custo.
4. **Explicar:** feche o código e ensine o assunto em voz alta, em dois minutos.
5. **Registrar:** marque o checkpoint apenas se conseguir explicar o motivo da decisão.

NOTA

Regra de honestidade profissional: diga “implementei e executei” para S3, Glue, PySpark, Parquet, Glue Data Catalog, Athena, Terraform e testes. Diga “implementei o código, mas não provisionei” para Lambda e Airflow. Diga “comparei conceitualmente” para Redshift, EMR, EC2, RDS, DynamoDB, Step Functions, streaming, Data Mesh, Snowflake e Great Expectations.

Mapa do laboratório

```
Open-Meteo API
-> src/ingest.py
-> S3 raw: JSON preservado
-> glue/weather_transform.py
-> S3 curated: Parquet particionado
-> Glue Data Catalog
-> Athena SQL
```

O Terraform descreve a infraestrutura. Os testes verificam contratos e falhas. Airflow e Lambda demonstram pontos possíveis de orquestração e reação a eventos, sem terem sido implantados.

Área	Arquivo principal	Pergunta que ela responde
Visão geral	README.md	Qual problema o projeto resolve?
Ingestão	src/ingest.py	Como a API vira dado RAW reproduzível?
Transformação	glue/weather_transform.py	Como o JSON vira dado tipado e confiável?
Spark	glue/spark_experiments.py	Onde partitions, shuffle, broadcast e skew aparecem?
Catálogo e SQL	sql/01_catalog.sql , sql/02_learning_queries.sql	Como arquivos viram tabelas consultáveis?

Área	Arquivo principal	Pergunta que ela responde
Infraestrutura	<code>terraform/</code>	Como reproduzir recursos AWS com controle?
Orquestração	<code>airflow/weather_pipeline_dag.py</code>	Como ordenar e recuperar as etapas?
Evento	<code>lambda/raw_event_validator.py</code>	O que pode reagir à chegada de um objeto?
Qualidade	<code>tests/</code> , transformação e ingestão	Como falhar antes de publicar dado incorreto?
Evidência AWS	<code>evidence/aws_run_summary.json</code>	O que foi realmente executado e medido?

Preparação segura

Você pode aprender quase todo o projeto sem recriar recursos na AWS. Primeiro estude localmente e consulte as evidências já publicadas. Só volte a executar Glue/Athena se quiser praticar operação cloud e aceitar novo custo.

Opção A: navegar no GitHub

Abra o repositório e use a ordem das sessões abaixo. Para encontrar um termo, pressione `/` no GitHub ou use a busca do repositório. Comece sempre pelo arquivo, não pela pasta inteira.

Opção B: executar localmente em Linux, WSL ou CloudShell

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -q
```

Execução local completa, que acessa a API pública e gera `data/`:

```
./scripts/run_local.sh
```

Experimentos Spark isolados:

```
spark-submit glue/spark_experiments.py
```

O que não executar agora

Não rode `cloudshell_execute.sh` apenas para estudar o código. O laboratório AWS já foi executado e está documentado. Quando terminar definitivamente de explorar os recursos existentes, revise o alvo antes de usar:

```
terraform -chdir=terraform destroy
```

Esse comando remove recursos e objetos do bucket do laboratório. Ele é deliberadamente uma etapa separada.

Sessão 1 - Conte a história antes de estudar os detalhes

Objetivo: entender o fluxo ponta a ponta, o grão do dado e os limites honestos do projeto.

Abra nesta ordem:

1. `README.md`
2. `evidence/aws_run_summary.json`
3. `docs/BUILD_LOG.md`

Faça: desenhe de memória cinco caixas: fonte, RAW, transformação, CURATED e consulta. Embaixo de cada uma, escreva o serviço ou arquivo responsável.

Observe:

- a fonte possui quatro cidades e dados horários;
- o grão é uma linha por `location_id` + `observed_at`;
- RAW preserva a resposta; CURATED aplica contrato e formato analítico;
- a execução AWS produziu 2.976 linhas e zero chaves duplicadas;
- Spark foi escolhido para aprendizado distribuído, não por ser necessário ao pequeno volume.

Explique em dois minutos:

NOTA

Construí um pipeline batch de clima. Python coleta a API e preserva JSON no S3 RAW. Um job AWS Glue com PySpark valida, tipa, deduplica e grava Parquet particionado na CURATED. O Glue Data Catalog fornece o schema e o Athena consulta os arquivos. A infraestrutura é reproduzível com Terraform e o comportamento é testado com pytest e CI.

Checkpoint:

- ☐ Consigo dizer qual é o grão sem consultar o README.
- ☐ Sei diferenciar dado no S3 de tabela no catálogo.
- ☐ Consigo apontar uma escolha feita para aprender e não para otimizar este volume.

Perguntas de revisão:

1. Por que o projeto tem duas camadas e não três?
2. O que seria necessário para adicionar uma camada Gold de verdade?
3. Quais resultados provam que o fluxo chegou ao fim?

Sessão 2 - Ingestão, RAW, contrato e idempotência

Objetivo: seguir uma chamada de API até o objeto RAW e entender como uma reexecução segura foi projetada.

Abra nesta ordem:

1. `src/ingest.py`
2. `tests/test_ingest.py`
3. `scripts/run_local.sh`

Procure no código: definição das localizações, parâmetros de data, envelope de metadados, caminho do objeto e comportamento de sobrescrita.

Faça: execute somente os testes de ingestão:

```
pytest -q tests/test_ingest.py
```

Depois responda: se a mesma cidade e o mesmo período forem processados duas vezes, o projeto cria um novo nome imprevisível ou escreve no mesmo destino lógico?

Conexões importantes:

- **RAW:** preserva payload e metadados para auditoria e reprocessamento.
- **Idempotência:** a mesma entrada deve levar ao mesmo estado útil, sem duplicar o resultado.
- **Batch:** um intervalo de datas é coletado como unidade de processamento.
- **S3:** é armazenamento de objetos; prefixos organizam dados, mas não são diretórios transacionais.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

A ingestão é pequena e orientada a I/O, então usei Python em vez de Spark. O RAW mantém a resposta original envelopada com metadados. O caminho é determinístico, permitindo reprocessamento idempotente. A transformação lê o RAW, por isso não precisa chamar a API novamente.

Checkpoint:

- ☐ Consigo mostrar onde a URL e as cidades são definidas.
- ☐ Consigo explicar a diferença entre retry e duplicação.
- ☐ Sei por que JSON continua adequado na borda, embora CURATED use Parquet.

Desafio opcional: adicione mentalmente uma quinta cidade e liste tudo o que deveria continuar funcionando sem alteração estrutural.

Sessão 3 - PySpark, qualidade e publicação CURATED

Objetivo: entender como arrays do JSON se tornam linhas tipadas e por que qualidade antecede a publicação.

Abra nesta ordem:

1. glue/weather_transform.py
2. tests/test_glue_transform.py
3. evidence/aws_run_summary.json

Procure no código: `require_raw_schema`, transformação horária, `casts`, `quality_metrics`, chave de deduplicação, escrita Parquet e particionamento por data.

Faça: execute:

```
pytest -q tests/test_glue_transform.py
```

Classifique cada regra como contrato, completude, validade ou unicidade:

- campos obrigatórios no envelope;
- `location_id` e `observed_at` não nulos;
- umidade entre 0 e 100;
- precipitação e vento não negativos;
- uma linha por localização e instante.

Conexões importantes:

- **Schema explícito** reduz surpresas de inferência e documenta o contrato.
- **Deduplicação** exige uma chave coerente com o grão.
- **Fail fast** impede que CURATED pareça confiável quando a validação falhou.
- **Glue Data Quality ou Great Expectations** poderiam centralizar e reportar regras; neste lab, regras PySpark e testes são suficientes e visíveis.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

No Glue, explodo as séries horárias, aplico tipos, valido domínios e unicidade e só então escrevo a CURATED. A chave de deduplicação coincide com o grão da fato. Se uma regra crítica falhar, o job para antes de publicar dado inválido. Uma evolução seria externalizar regras e observabilidade em Glue Data Quality ou Great Expectations.

Checkpoint:

- ☐ Consigo reconstruir a chave da fato.
- ☐ Sei distinguir corrigir duplicatas de esconder a causa delas.
- ☐ Consigo explicar por que testes de código e monitoramento de dados são complementares.

Sessão 4 - O motor Spark: lazy, partitions, shuffle, broadcast, skew e small files

Objetivo: ligar conceitos distribuídos ao plano físico e a experimentos observáveis.

Abra: `glue/spark_experiments.py` e as seções correspondentes de `docs/LEARNING.md`.

Faça: execute e salve as linhas que contêm `Exchange`, `BroadcastExchange` e `BroadcastHashJoin`:

```
spark-submit glue/spark_experiments.py
```

Roteiro de observação:

Conceito	O que procurar	Interpretação
Lazy evaluation	plano antes de <code>count()</code>	transformations descrevem; actions executam
Partition	4, depois 8, depois 2	unidade de paralelismo e, muitas vezes, uma task por estágio
Shuffle	<code>Exchange</code>	dados cruzam executors para reunir chaves
Broadcast	<code>BroadcastExchange</code>	lookup pequeno é enviado aos executors
Skew	chave <code>hot</code> com 90%	uma task pode virar gargalo
Small files	24 arquivos contra 2	metadados e agendamento podem dominar o trabalho

Explique as relações:

- `groupBy`, `join` e `repartition` podem provocar shuffle;
- filtrar linhas e selecionar colunas cedo reduz bytes movimentados;
- broadcast evita shuffle da relação grande, mas consome memória em cada executor;
- mais partitions não corrigem uma chave extremamente concentrada;
- compactar ajuda contra small files, mas arquivos gigantes reduzem paralelismo.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

Eu não trato shuffle como algo sempre ruim; ele é necessário em operações globais. Procuro reduzir dados antes dele e evitar redistribuições redundantes. Para dimensão pequena, broadcast pode evitar mover a fato. Verifico o plano físico e métricas antes de aplicar tuning. Também observo skew e tamanho dos arquivos, porque somente aumentar partitions não resolve distribuição desigual.

Checkpoint:

- ☐ Consigo apontar a action que dispara o plano lazy.
- ☐ Sei quando `repartition` e `coalesce` tendem a ser usados.
- ☐ Consigo explicar por que broadcast tem um limite prático de memória.

- ☐ Não apresento os tempos locais como benchmark universal.

Sessão 5 - Parquet, catálogo, particionamento e Athena

Objetivo: entender como layout físico, metadados e SQL afetam custo e desempenho.

Abra nesta ordem:

1. `sql/01_catalog.sql`
2. `sql/02_learning_queries.sql`
3. `src/run_athena.py`
4. `evidence/aws_run_summary.json`

Faça sem novo custo: compare no resumo de evidências:

- consulta do mês: 13.426 bytes;
- consulta de um dia: 457 bytes;
- redução observada: 96,6%.

Explique a cadeia causal:

1. PySpark escreve Parquet colunar.
2. A fato é organizada por `year/month/day`.
3. O catálogo declara schema, localização e partitions.
4. O filtro explícito nas colunas de partition permite partition pruning.
5. Athena lê menos objetos e bytes, reduzindo tempo e custo por scan.

Não confunda:

- **Column pruning:** lê apenas colunas solicitadas.
- **Predicate pushdown:** usa filtros/estatísticas para evitar blocos incompatíveis.
- **Partition pruning:** ignora prefixos/partitions inteiras.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

Escolhi Parquet porque o consumo é analítico e consulta subconjuntos de colunas. Particionei por dia porque data é filtro natural e tem cardinalidade aceitável. Na execução, o filtro de uma partition reduziu o scan em 96,6%. O catálogo não move os dados; ele fornece schema e localização para o Athena interpretá-los.

Checkpoint:

- ☐ Consigo diferenciar S3, Data Catalog e Athena em uma frase cada.
- ☐ Sei por que não particionar por timestamp horário ou ID de alta cardinalidade.
- ☐ Consigo explicar por que `count(*)` pode ser uma demonstração ruim de scan.

Sessão 6 - Modelagem dimensional e escolha do serviço de dados

Objetivo: conectar a forma dos dados ao tipo de workload.

Abra: definição de `fact_weather_hourly` e `dim_location` em `sql/01_catalog.sql`, depois a consulta de join em `sql/02_learning_queries.sql`.

Faça: para cada coluna, decida se é chave, dimensão descritiva ou medida. Depois compare:

Conceito ou serviço	Melhor encaixe	Papel neste laboratório
Star schema	análise simples, poucos joins	fato + dimensão implementadas
Snowflake schema	dimensões normalizadas e governadas	apenas comparação conceitual
OLTP / RDS	transações, integridade, estado operacional	não necessário
DynamoDB	acesso por chave e baixa latência em escala	não necessário
Athena	consulta ad hoc sobre arquivos	implementado e executado
Redshift	warehouse, concorrência e workloads repetitivos	apenas comparação conceitual
Snowflake	warehouse SaaS multi-cloud	apenas comparação conceitual

Exercício de arquitetura: escolha um serviço para cada cenário:

1. Checkout de uma loja com transações relacionais.
2. Consulta de sessão por chave em poucos milissegundos.
3. Investigação SQL esporádica sobre arquivos no S3.
4. Centenas de dashboards com SLA previsível.

Respostas esperadas: RDS, DynamoDB, Athena e um warehouse como Redshift, respectivamente, desde que requisitos reais confirmem a escolha.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

A tecnologia segue o workload. Este projeto é analítico e esporádico, então S3 mais Athena evita um cluster permanente. RDS serviria transações relacionais; DynamoDB, padrões de acesso por chave e baixa latência; Redshift ganha força com serving analítico recorrente, concorrência e SLAs. Não provisionei esses serviços apenas para marcar tecnologias.

Checkpoint:

- ☐ Consigo explicar o grão antes de falar das colunas.
- ☐ Sei diferenciar star de snowflake schema e Snowflake, o produto.
- ☐ Consigo escolher serviço a partir do workload, sem começar pela marca.

Sessão 7 - Terraform, orquestração, CI e FinOps

Objetivo: entender como o pipeline é reproduzido, coordenado, testado e controlado financeiramente.

Abra nesta ordem:

1. terraform/main.tf, variables.tf e outputs.tf
2. scripts/cloudshell_prepare.sh e scripts/cloudshell_execute.sh
3. airflow/weather_pipeline_dag.py
4. lambda/raw_event_validator.py
5. .github/workflows/tests.yml

Faça: execute localmente:

```
terraform -chdir=terraform fmt -check
terraform -chdir=terraform validate
pytest -q
```

Se o Terraform não estiver instalado, leia o plano registrado em [docs/BUILD_LOG.md](#); não o instale apenas para marcar o checkpoint.

Observe:

- Terraform cria 9 recursos e mantém estado; **plan** separa revisão de **apply**.
- IAM deve conceder somente ações necessárias.
- Airflow descreve dependências, retries e ordem, mas não processa os dados pesados.
- Lambda valida um evento curto; Glue transforma o dataset.
- Step Functions seria alternativa AWS nativa para uma máquina de estados gerenciada.
- GitHub Actions executa testes a cada mudança relevante.
- O workgroup Athena limita bytes por consulta e não há compute permanente.

FinOps aplicado:

- Parquet e pruning reduzem scan do Athena;
- Glue usa somente dois workers durante o job;
- serviços caros e permanentes foram excluídos;
- execução cloud exige confirmação explícita;
- destruição dos recursos ficou documentada e separada.

Resposta de entrevista:

RESPOSTA DE ENTREVISTA

Usei Terraform para revisar e reproduzir recursos, com uma etapa de plan antes do apply. Separei orquestração de processamento: Airflow ou Step Functions coordenariam, enquanto Glue executa Spark. CI valida o código, e FinOps aparece no formato colunar, partition pruning, limite do workgroup, ausência de compute permanente e confirmação explícita de custo.

Checkpoint:

- ☐ Consigo explicar **plan**, **apply**, **state** e **destroy**.
- ☐ Sei diferenciar orquestrador de motor de processamento.
- ☐ Consigo citar três decisões concretas de custo.

☐ Sei que o DAG e a Lambda não foram implantados.

Sessão 8 - Síntese, DataOps e simulação de entrevista

Objetivo: transformar arquivos e conceitos em uma narrativa profissional verificável.

Abra: [docs/LEARNING.md](#), os testes e o histórico do GitHub Actions.

Faça três apresentações, sem ler:

Versão de 30 segundos

Problema, fluxo principal e resultado mensurável.

Versão de 2 minutos

Inclua grão, por que Python + PySpark, RAW/CURATED, qualidade, Parquet/partitions e evidência Athena.

Versão de 5 minutos

Inclua trade-offs, falhas, idempotência, IaC, CI, custo e o que você mudaria em produção.

Perguntas para simulação:

1. Por que usar Spark se o dataset é pequeno?
2. Como o pipeline se comporta ao ser reexecutado?
3. Onde pode ocorrer shuffle e como você o investigaria?
4. Como você provaria que partition pruning funcionou?
5. O que acontece se uma cidade vier sem um campo obrigatório?
6. Por que Athena e não Redshift?
7. Quando Airflow seria preferível a Step Functions?
8. Quais recursos realmente foram implantados?
9. Como você evoluiria qualidade e observabilidade?
10. O que mudaria para streaming?

Checklist da resposta forte:

- começa pelo requisito, não pela tecnologia;
- aponta o arquivo ou a evidência;
- apresenta uma alternativa;
- explica o trade-off;
- não inventa escala, produção ou serviços não executados.

Resposta sobre evolução:

RESPOSTA DE ENTREVISTA

Em produção eu adicionaria observabilidade e alertas, política formal de schema, qualidade com métricas históricas, catálogo governado, estratégia de backfill, segurança por ambientes e SLOs. Consideraria Iceberg para evolução e transações de tabela, Glue Data Quality ou Great Expectations para regras reutilizáveis e um orquestrador gerenciado conforme a complexidade. Essas seriam decisões orientadas por requisitos, não extensões automáticas.

Checkpoint final:

- ☐ Consigo apresentar o projeto em 2 minutos sem abrir o repositório.
- ☐ Respondo “por quê?” para cada serviço escolhido.
- ☐ Cito números reais: 2.976 linhas, 0 duplicadas e 96,6% de redução de scan.
- ☐ Separo implementado, código opcional e comparação conceitual.
- ☐ Tenho pelo menos uma melhoria futura priorizada e justificável.

Conceitos adjacentes: estude sem aumentar o projeto agora

Estes assuntos são relevantes, mas não precisam virar novos recursos pagos neste laboratório.

EC2, EMR e Glue

- **EC2:** máquina virtual generalista; oferece controle e exige mais operação.
- **EMR:** plataforma de big data com maior controle sobre clusters e frameworks.
- **Glue:** serviço gerenciado/serverless de integração de dados; reduz operação para jobs episódicos.

Pergunta-chave: o workload exige controle contínuo do cluster ou execução gerenciada sob demanda?

Medallion e Data Mesh

- **Medallion:** organização técnica por estágios de refinamento dentro de uma plataforma de dados.
- **Data Mesh:** abordagem organizacional de domínio, produto de dados, governança federada e plataforma self-service.

Eles não são alternativas diretas. Este laboratório demonstra camadas; não demonstra uma organização Data Mesh.

Batch e streaming

O laboratório é batch porque a fonte e o objetivo aceitam processamento por intervalo. Para streaming, seria necessário definir eventos, atraso aceitável, ordenação, duplicatas, watermark, estado e recuperação. Adicionar Kafka/Kinesis sem esse requisito só aumentaria complexidade.

DevOps, DataOps e CI/CD

- **DevOps:** colaboração, automação e operação confiável de software e infraestrutura.
- **DataOps:** aplica esses princípios ao ciclo de dados, incluindo contratos, qualidade, linhagem e observabilidade.
- **CI:** neste projeto, executa testes automaticamente.
- **CD:** não foi configurado para aplicar Terraform automaticamente, pois mudanças cloud exigem revisão e controle de custo.

Plano de retenção após as oito sessões

Não releia tudo. Use recuperação ativa:

- **24 horas depois:** redesenhe a arquitetura e responda cinco perguntas sem consultar.
- **3 dias depois:** execute os testes e explique uma falha simulada.
- **7 dias depois:** faça a apresentação de 5 minutos e grave o áudio.
- **14 dias depois:** peça a alguém, ou a uma IA, uma entrevista técnica baseada no repositório.
- **30 dias depois:** escolha uma evolução pequena, como adicionar uma regra de qualidade e seu teste.

Navegação rápida por intenção

Quero revisar...	Abra primeiro	Depois valide com...
arquitetura	<code>README.md</code>	<code>evidence/aws_run_summary.json</code>
Python/API/idempotência	<code>src/ingest.py</code>	<code>tests/test_ingest.py</code>
PySpark/Data Quality	<code>glue/weather_transform.py</code>	<code>tests/test_glue_transform.py</code>
performance Spark	<code>glue/spark_experiments.py</code>	saída de <code>spark-submit</code>
Parquet/partitions/Athena	<code>sql/02_learning_queries.sql</code>	resumo de scan em <code>evidence/</code>
modelagem dimensional	<code>sql/01_catalog.sql</code>	query de join
AWS/IaC/FinOps	<code>terraform/</code>	<code>docs/BUILD_LOG.md</code>
orquestração	DAG Airflow	Lambda e comparação Step Functions
confiabilidade	<code>tests/</code>	GitHub Actions
respostas conceituais	<code>docs/LEARNING.md</code>	apresentação sem consulta

Critério de conclusão

Você concluiu o estudo inicial quando consegue, sem abrir os arquivos:

1. desenhar o fluxo e nomear a responsabilidade de cada componente;
2. explicar três decisões e suas alternativas;
3. demonstrar onde qualidade, idempotência e custo aparecem;
4. interpretar [Exchange](#) e [BroadcastHashJoin](#) em um plano Spark;
5. citar uma evidência real da AWS e uma limitação honesta;
6. apresentar o projeto em dois minutos e responder cinco perguntas de aprofundamento.

Depois disso, use [docs/LEARNING.md](#) como referência detalhada, não como texto para decorar.